41039 Programming 1

Revision of Week 11: designing object-oriented programs



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 10 questions, about 35 minutes

#	Type	Topic
Q1	Fill in the blanks	Experts, cohesion, coupling, YAGNI
Q2	Multiple choice	Which `Basket` needs no edit?
Q3	Multiple choice	Which `Tally` breaks the promise?
Q4	Predict the output	Library fines (Java)
Q5	Predict the output	A screen and a recorder (Python)
Q6	Find the bug	The library member (Java, 4 bugs)
Q7	Find the bug	The checkout (Python, 3 bugs)
Q8	Write the code	Ask the expert: a cart (Java)
Q9	Write the code	One pass mark in one place (Python)
Q10	Write the code	Two desks, one stock (Java)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. The answer appears on the **next click**.

Week 11 in five ideas (read this first)

1. **One responsibility.** One clear job, so one reason to change. Keep **calculating** apart from **printing**: `total()` returns a number; other code shows it.
2. **Information Expert.** Put a calculation in the class that *has* the data. `Order` **asks** each `Line` for its subtotal; it never reaches into its `private` fields.
3. **Depend on an interface, and be given it.** `Basket` keeps a `PriceRule` passed to its constructor (Python: pass a function). A new rule is a new class; `Basket` does not change.
4. **Keep the promise.** A replacement must *behave* as callers expect, not just share method names. Ask only for what you use (e.g. just `send`).
5. **Cohesion and coupling.** High cohesion: parts serve one purpose. Low coupling: few details of others. **DRY**: one rule in one place. **YAGNI**: no code for imagined futures.

```
int total() {    // in class Order
    int sum = 0;
    for (Line l : lines)
        sum += l.subtotal();  // ask
    return sum;               // no print
}
```

Q1 Fill in the blanks Responsibilities and dependencies

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- Put a calculation in the class that holds its data: Information **(1)** .
- All parts serve one purpose: high **(2)** . Few details of other classes needed: low **(3)** .
- `PriceRule` is an **(4)** : any rule class fits.
- A replacement must keep the **(5)** callers rely on, not just the method names.
- Code for a feature nobody asked for breaks **(6)** .

Word bank (2 min, alone)

a) cohesion	**f)** interface
b) DRY	**g)** controller
c) expert	**h)** YAGNI
d) behaviour	**i)** coupling
e) static	**j)** inheritance

Q1 Fill in the blanks Responsibilities and dependencies

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- Put a calculation in the class that holds its data: Information **(1)** .
- All parts serve one purpose: high **(2)** . Few details of other classes needed: low **(3)** .
- `PriceRule` is an **(4)** : any rule class fits.
- A replacement must keep the **(5)** callers rely on, not just the method names.
- Code for a feature nobody asked for breaks **(6)** .

Answers

(1) c) expert
(2) a) cohesion **(3)** i) coupling
(4) f) interface **(5)** d) behaviour
(6) h) YAGNI

Not used: b) DRY, e) static, g) controller, j) inheritance.

DRY = one rule in one place. A **controller** coordinates a request but leaves each rule to its expert.

Q2 Multiple choice Which Basket needs no edit?

Given: `StudentPrice` (80%) and the **new** `Half` implement the interface `PriceRule`.

Version 1

```
class Basket {
    int price(int c,
              boolean student) {
        if (student)
            return c * 80 / 100;
        return c;
    }
}
```

Version 2

```
class Basket {
    private PriceRule rule =
        new StudentPrice();
    int price(int c) {
        return rule.apply(c);
    }
}
```

Version 3

```
class Basket {
    private PriceRule rule;
    Basket(PriceRule r) {
        rule = r;
    }
    int price(int c) {
        return rule.apply(c);
    }
}
```

Choose one (90 s)

Which can use `Half` with **no** edit to `Basket`?

A. V1 only **B.** V2 only
C. V3 only **D.** V2, V3

Hint: the code that creates a basket is **outside** the class. What can it hand to each version?

Q2 Multiple choice Which Basket needs no edit?

Given: StudentPrice (80%) and the **new** Half implement the interface PriceRule.

Version 1

```
class Basket {
    int price(int c,
              boolean student) {
        if (student)
            return c * 80 / 100;
        return c;
    }
}
```

Version 2

```
class Basket {
    private PriceRule rule =
        new StudentPrice();
    int price(int c) {
        return rule.apply(c);
    }
}
```

Version 3

```
class Basket {
    private PriceRule rule;
    Basket(PriceRule r) {
        rule = r;
    }
    int price(int c) {
        return rule.apply(c);
    }
}
```

Choose one (90 s)

Which can use Half with **no** edit to Basket?

A. V1 only **B.** V2 only

✓ **C.** V3 only **D.** V2, V3

Answer: C

V3 ✓ the caller writes new Basket(new Half()): 500.

V1 ✗ knows only "student or not": a new rule is new code.

V2 ✗ (so not D) **picks** StudentPrice itself: to use Half, edit Basket.

Q3 Multiple choice Which `Tally` breaks the promise?

```
interface Tally {
    // Promise: total() is the sum of
    // every n given to add(n).
    void add(int n);
    int total();
}
// in class Demo: works with ANY Tally
static int check(Tally t) {
    t.add(5);
    t.add(3);
    return t.total();
}
```

Each option is a class that `implements Tally`; its `total()` returns what it keeps (B adds up its list). All four **compile**. Exactly **one** breaks `check`.

Choose one (60 seconds)

A. `Sum`: keeps a running sum, `s += n`.
B. `ListTally`: keeps every `n` in a list.
C. `LastTally`: keeps only the last `n`.
D. `CountTally`: `Sum` plus a new `count()`.

Q3 Multiple choice Which `Tally` breaks the promise?

```
interface Tally {
    // Promise: total() is the sum of
    // every n given to add(n).
    void add(int n);
    int total();
}
// in class Demo: works with ANY Tally
static int check(Tally t) {
    t.add(5);
    t.add(3);
    return t.total();
}
```

Each option is a class that `implements Tally`; its `total()` returns what it keeps (B adds up its list). All four **compile**. Exactly **one** breaks `check`.

Choose one (60 seconds)

A. `Sum`: keeps a running sum, `s += n`.
B. `ListTally`: keeps every `n` in a list.
✓ **C.** `LastTally`: keeps only the last `n`.
D. `CountTally`: `Sum` plus a new `count()`.

Answer: C

`check` gives 8 for A, B, D but **3** for C: it forgets the 5. Same method names, broken promise. The compiler checks names, not behaviour.

B stores numbers differently and **D** adds a method: `check` cannot tell.

Q4 Predict the output Library fines

```
interface Fee { int charge(int days); }
class Flat implements Fee {
    public int charge(int d) { return 5; }
}
class Daily implements Fee {
    public int charge(int d) { return 2 * d; }
}
class Library {
    private final Fee fee;
    Library(Fee fee) { this.fee = fee; }
    int fine(int daysLate) {
        if (daysLate <= 0) return 0;
        return fee.charge(daysLate);
    }
}
// in main, in another class Demo
Library a = new Library(new Daily());
Library b = new Library(new Flat());
System.out.println(a.fine(3) + " " + b.fine(3));
System.out.println(a.fine(0) + " " + b.fine(9));
```

Predict (90 seconds, alone)

1. Write down the **two** lines that are printed.

2. The library wants a new fee: 1 per day, at most 4. Which existing class must you edit?

Q4 Predict the output Library fines

```
interface Fee { int charge(int days); }
class Flat implements Fee {
    public int charge(int d) { return 5; }
}
class Daily implements Fee {
    public int charge(int d) { return 2 * d; }
}
class Library {
    private final Fee fee;
    Library(Fee fee) { this.fee = fee; }
    int fine(int daysLate) {
        if (daysLate <= 0) return 0;
        return fee.charge(daysLate);
    }
}
// in main, in another class Demo
Library a = new Library(new Daily());
Library b = new Library(new Flat());
System.out.println(a.fine(3) + " " + b.fine(3));
System.out.println(a.fine(0) + " " + b.fine(9));
```

Answers

1. `6 5`
`0 5`

Each library uses the fee it was **given**: `Daily` gives $2 \times 3 = 6$; `Flat` is always 5. For 0 days `Library` returns 0 *before* it asks the fee.

2. None. Write a new class

```
class Capped implements Fee {
  public int charge(int d)
    { return Math.min(d, 4); } }
```

and pass `new Capped()`. `fine(3)`, `fine(9)`, `fine(0)` give `3 4 0`. The "0 if not late" rule stays in `Library`.

Q5 Predict the output A screen and a recorder

```
class Screen:
    def send(self, text):
        print("screen:", text)

class Recorder:
    def __init__(self):
        self.sent = []
    def send(self, text):
        self.sent.append(text)

def remind(channel, names):
    for name in names:
        if name != "":
            channel.send("Due: " + name)

remind(Screen(), ["An", ""])
r = Recorder()
remind(r, ["Binh", "Chi"])
print(len(r.sent), r.sent[0])
```

Predict (90 seconds, alone)

1. Write down the **two** lines that are printed.

2. `remind` only ever calls `send`. Why does that make `remind` easy to test?

Q5 Predict the output A screen and a recorder

```python
class Screen:
    def send(self, text):
        print("screen:", text)

class Recorder:
    def __init__(self):
        self.sent = []
    def send(self, text):
        self.sent.append(text)

def remind(channel, names):
    for name in names:
        if name != "":
            channel.send("Due: " + name)

remind(Screen(), ["An", ""])
r = Recorder()
remind(r, ["Binh", "Chi"])
print(len(r.sent), r.sent[0])
```

Answers

1. `screen: Due: An`
`2 Due: Binh`

The empty name is skipped. `Screen` prints at once. `Recorder` prints **nothing**: it keeps both messages in its list.

2. `remind` needs **only** an object with `send`. A test passes a `Recorder`, then checks `r.sent`: nothing to read off a screen. Asking for one small capability keeps `remind` loosely coupled.

Q6 Find the bug The library member (Java)

```java
import java.util.*;
class Loan {
    private final int due;
    Loan(int due) { this.due = due; }
    boolean isLate(int day) { return day >= due; }
}
public class Member {
    private List<Loan> loans = new ArrayList<>();
    List<Loan> getLoans() { return loans; }
    boolean borrow(Loan l) {
        if (loans.size() > 3) return false;
        loans.add(l);
        return true;
    }
    int countLate(int day) {
        int n = 0;
        for (Loan l : loans) if (day > l.due) n++;
        return n;
    }
}
```

✗ Find 4 bugs (3 min): 1 compile error + 3 logic errors. Give line and fix.

What the classes should do

- A loan is late only **after** its due day
- Max **3** loans; `getLoans()` gives a **copy**

Test (`main` in another class)

```java
Member m = new Member();
m.borrow(new Loan(10));
m.borrow(new Loan(12));
System.out.println(m.countLate(10));
m.borrow(new Loan(20));
System.out.println(m.borrow(new Loan(20)));
m.getLoans().clear();
System.out.println(m.getLoans().size());
```

Expected output

```
0
false
3
```

Q6 Find the bug The library member (Java)

```java
import java.util.*;
class Loan {
    private final int due;
    Loan(int due) { this.due = due; }
    boolean isLate(int day) { return day >= due; }
}
public class Member {
    private List<Loan> loans = new ArrayList<>();
    List<Loan> getLoans() { return loans; }
    boolean borrow(Loan l) {
        if (loans.size() > 3) return false;
        loans.add(l);
        return true;
    }
    int countLate(int day) {
        int n = 0;
        for (Loan l : loans) if (day > l.due) n++;
        return n;
    }
}
```

✗ Find 4 bugs (3 min): 1 compile error + 3 logic errors. Give line and fix.

✓ The 4 fixes

- **Line 17** (compile error: `due has private access in Loan`): `Member` reaches into `Loan`. Ask the expert: `if (l.isLate(day)) n++;`
- **Line 5**: `>=` makes a loan late *on* its due day. Use `>`.
- **Line 11**: `>` lets a 4th loan in. Use `>=`.
- **Line 9**: hands out the **real** list, so outside code can clear it. Return a copy: `new ArrayList<>(loans)`.

Fix only line 17: the test prints `1`, `true`, `0`.

Q7 Find the bug The checkout (Python)

```
1  def regular(cents):
2      return cents
3  def student(cents):
4      return cents * 80 // 100
5
6  class Checkout:
7      def __init__(self, rule):
8          self.rule = regular
9      def charge(self, cents):
10         print(self.rule(cents))
11     def receipt(self, cents):
12         return "Paid: " + self.charge(cents)
13
14 c = Checkout(student)
15 print(c.receipt(1000))
16 print(c.receipt(505))
```

✗ **Find 3 bugs** in lines 6–12 (3 min): 2 logic errors + 1 crash. Give each line number and its fix.

What `Checkout` should do

- Use the rule function it is **given**
- `charge(cents)`: **return** the price; print nothing
- `receipt(cents)`: return `"Paid: <price>"`

Expected output

```
Paid: 800
Paid: 404
```

✗ Actual output (crashes at line 12)

```
1000
TypeError: can only concatenate str (not "NoneType") to str
```

Q7 Find the bug The checkout (Python)

```
1  def regular(cents):
2      return cents
3  def student(cents):
4      return cents * 80 // 100
5
6  class Checkout:
7      def __init__(self, rule):
8          self.rule = regular
9      def charge(self, cents):
10         print(self.rule(cents))
11     def receipt(self, cents):
12         return "Paid: " + self.charge(cents)
13
14 c = Checkout(student)
15 print(c.receipt(1000))
16 print(c.receipt(505))
```

✗ Find 3 bugs in lines 6–12 (3 min): 2 logic errors + 1 crash. Give each line number and its fix.

✓ The 3 fixes

- **Line 8**: ignores the `rule` it was given and picks `regular` itself, so the student pays 1000. Use `self.rule = rule`.
- **Line 10**: a calculation that **prints** gives back `None`, so no other code can use the price. Use `return self.rule(cents)`.
- **Line 12**: `"Paid: " + a number` crashes. Fix only line 10 and it still fails with `(not "int")`. Use `"Paid: " + str(self.charge(cents))`.

Q8 Write the code Ask the expert: a cart (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `cost()`: the item's price in cents times its quantity.
- `total()`: the sum of every item's cost. `Cart` must **ask** each item: `cents` and `qty` are `private`.

Skeleton

```
class Item {
    private final int cents, qty;
    Item(int c, int q) { cents = c; qty = q; }
    int cost() { return 0; }  // TODO
}
class Cart {
    private List<Item> items = new ArrayList<>();
    void add(Item i) { items.add(i); }
    int total() {
        return 0;  // TODO
    }
}
```

Test (in `main`)

```
Cart c = new Cart();
System.out.println(c.total());
c.add(new Item(250, 2));
c.add(new Item(120, 5));
c.add(new Item(999, 0));
System.out.println(c.total());
```

Must print

```
0
1100
```

Q8 Write the code Ask the expert: a cart (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `cost()`: the item's price in cents times its quantity.
- `total()`: the sum of every item's cost. `Cart` must **ask** each item: `cents` and `qty` are `private`.

Skeleton

```
class Item {
    private final int cents, qty;
    Item(int c, int q) { cents = c; qty = q; }
    int cost() { return 0; }  // TODO
}
class Cart {
    private List<Item> items = new ArrayList<>();
    void add(Item i) { items.add(i); }
    int total() {
        return 0;  // TODO
    }
}
```

✓ One answer

```
class Item {
    private final int cents, qty;
    Item(int c, int q) { cents = c; qty = q; }
    int cost() { return cents * qty; }
}
class Cart {
    private List<Item> items = new ArrayList<>();
    void add(Item i) { items.add(i); }
    int total() {
        int sum = 0;
        for (Item i : items) sum += i.cost();
        return sum;
    }
}
```

Q9 Write the code One pass mark in one place (Python)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `__init__`: keep the pass mark and an empty list. `add(mark)`: add one mark.
- `passes(mark)`: is `mark` $\geq$ the pass mark? The two counts must **not** read the pass mark.

Skeleton

```
class Gradebook:
    def __init__(self, pass_mark):
        pass  # TODO
    def add(self, mark):
        pass  # TODO
    def passes(self, mark):
        return False  # TODO
    def count_passes(self):
        return 0  # TODO
    def count_fails(self):
        return 0  # TODO
```

Test

```
g = Gradebook(50)
for m in [49, 50, 85]:
    g.add(m)
print(g.count_passes(), g.count_fails())
h = Gradebook(60)
h.add(50)
print(h.count_passes(), h.count_fails())
```

Must print

```
2 1
0 1
```

Q9 Write the code One pass mark in one place (Python)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `__init__`: keep the pass mark and an empty list. `add(mark)`: add one mark.
- `passes(mark)`: is `mark` $\geq$ the pass mark? The two counts must **not** read the pass mark.

Skeleton

```
class Gradebook:
    def __init__(self, pass_mark):
        pass  # TODO
    def add(self, mark):
        pass  # TODO
    def passes(self, mark):
        return False  # TODO
    def count_passes(self):
        return 0  # TODO
    def count_fails(self):
        return 0  # TODO
```

✓ One answer

```
class Gradebook:
    def __init__(self, pass_mark):
        self._pass_mark = pass_mark
        self._marks = []
    def add(self, mark):
        self._marks.append(mark)
    def passes(self, mark):
        return mark >= self._pass_mark
    def count_passes(self):
        n = 0
        for m in self._marks:
            if self.passes(m):
                n += 1
        return n
    def count_fails(self):
        return len(self._marks) - self.count_passes()
```

Q10 Write the code Two desks, one stock (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `take()`: at 0, return `false`, change nothing; else take one, return `true`.
- `Desk` keeps the `Stock` it is **given** (no count of its own) and uses it in `borrow`.

Skeleton

```
class Stock {
    private int count;
    Stock(int n) { count = n; }
    boolean take() {
        return false;  // TODO
    }
}
class Desk {
    // TODO: the Stock this desk uses
    Desk(Stock stock) { /* TODO */ }
    String borrow(String name) {
        return "";  // TODO
    }
}
```

Test (in `main`)

```
Stock books = new Stock(1);
Desk front = new Desk(books);
Desk back = new Desk(books);
System.out.println(front.borrow("An"));
System.out.println(back.borrow("Binh"));
System.out.println(books.take());
```

Must print

```
An borrowed
Binh: none left
false
```

Q10 Write the code Two desks, one stock (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `take()`: at 0, return `false`, change nothing; else take one, return `true`.
- `Desk` keeps the `Stock` it is **given** (no count of its own) and uses it in `borrow`.

Skeleton

```
class Stock {
    private int count;
    Stock(int n) { count = n; }
    boolean take() {
        return false;  // TODO
    }
}
class Desk {
    // TODO: the Stock this desk uses
    Desk(Stock stock) { /* TODO */ }
    String borrow(String name) {
        return "";  // TODO
    }
}
```

✓ One answer

```
class Stock {
    private int count;
    Stock(int n) { count = n; }
    boolean take() {
        if (count == 0) return false;
        count--;
        return true;
    }
}
class Desk {
    private final Stock stock;
    Desk(Stock stock) { this.stock = stock; }
    String borrow(String name) {
        if (stock.take()) return name + " borrowed";
        return name + ": none left";
    }
}
```

Looking ahead: one function that does every job

Every job in one function

```
rows = ["An,50", "Binh,70",
        "Chi,abc", "Dung,90"]

def report(lines):
    total = 0
    for line in lines:
        name, mark = line.split(",")
        total += int(mark)
    print("Average:", total / len(lines))

report(rows)
```

Think (1 minute)

Row `"Chi,abc"` crashes it with a `ValueError`. Split `report` into jobs. Which job should deal with a bad row?

Looking ahead: one function that does every job

Every job in one function

```
rows = ["An,50", "Binh,70",
        "Chi,abc", "Dung,90"]

def report(lines):
    total = 0
    for line in lines:
        name, mark = line.split(",")
        total += int(mark)
    print("Average:", total / len(lines))

report(rows)
```

Think (1 minute)

Row `"Chi,abc"` crashes it with a `ValueError`. Split `report` into jobs. Which job should deal with a bad row?

The **parse** job: text in, a mark (or `None`) out. Adding and printing never see bad text:

```
def parse(line):      # text -> a mark, or None
    name, mark = line.split(",")
    if mark.isdigit():
        return int(mark)
    return None

marks = []
for line in rows:
    m = parse(line)
    if m is None:
        print("Skipped:", line)
    else:
        marks.append(m)
print("Average:", sum(marks) / len(marks))
```

Prints `Skipped: Chi,abc` and then `Average: 70.0`. Today: review and integration, whole programs built from stages.

You should now be able to...

Who does what

- ✓ give a calculation to the class that has the data, and **ask** it (Q1, Q6, Q8)
- ✓ keep calculating apart from printing: return, do not print (Q7)
- ✓ keep one rule in one place (DRY) and avoid speculative code (YAGNI) (Q1, Q4, Q9)
- ✓ let a controller coordinate while the expert owns the rule and its state (Q10)

Depending on as little as possible

- ✓ depend on an interface or function you are **given** (Q2, Q4, Q7)
- ✓ tell a safe replacement from one that only matches the method names (Q3)
- ✓ ask a caller for only the capability it uses (Q5)
- ✓ protect an object's data: no field access from outside, no leaked lists (Q6)

Next: Week 12 – Review and integration

Tracing state accurately, a small data pipeline (read, parse, validate, model, summarise, report) with a clear error policy, testing the boundaries, and a first look at streams.